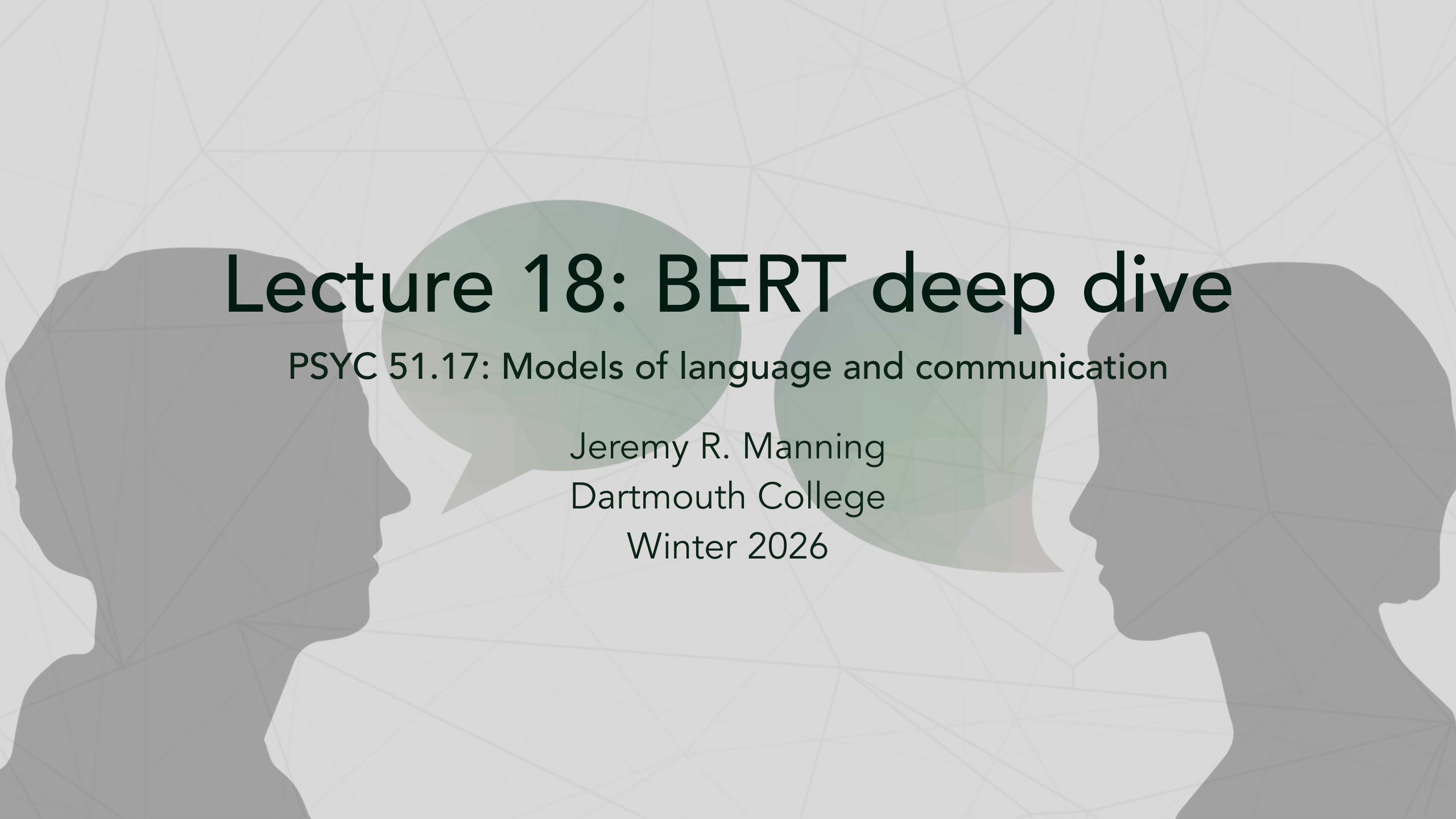




# Lecture 18: BERT deep dive

PSYC 51.17: Models of language and communication

Jeremy R. Manning  
Dartmouth College  
Winter 2026

# Learning objectives

By the end of this lecture, you will be able to...

1. Explain why bidirectional context matters for language understanding
2. Describe the masked language modeling (MLM) training objective and the 80/10/10 strategy
3. Compare BERT-Base and BERT-Large architectures
4. Understand the pre-train then fine-tune paradigm
5. Demonstrate how contextual embeddings handle polysemy

# What is BERT?

## Bidirectional Encoder Representations from Transformers

**BERT** is an **encoder-only transformer** that reads text in both directions simultaneously. Unlike autoregressive models (GPT), BERT sees the full context — both left and right — when processing each token.

The key innovation is **Masked Language Modeling (MLM)**: randomly mask tokens in the input and train the model to predict them from surrounding context.

## Analogy

Traditional language models read a sentence like filling in the end of a sentence. BERT is more like a fill-in-the-blank test — you can use clues from both sides.

# Why BERT was revolutionary

BERT changed everything about NLP (2018)

## Before BERT:

- Feature-based approaches (Word2Vec/GloVe as fixed features)
- Task-specific architectures for each problem
- Limited transfer learning — models trained from scratch
- Unidirectional or shallow bidirectional models (ELMo)

## BERT's contributions:

1. **Deep bidirectionality** — true bidirectional context at every layer, not just concatenating left-to-right and right-to-left
2. **Pre-train + fine-tune paradigm** — one pre-trained model adapted to many tasks with minimal changes
3. **State-of-the-art results** — beat previous best on 11 NLP benchmarks, sometimes by 10+ points

# Masked language modeling

## BERT's primary pre-training objective

### Training procedure:

1. Take a sentence from the training corpus
2. Randomly select 15% of tokens for prediction
3. Of those selected tokens:
  - **80%** are replaced with [MASK]
  - **10%** are replaced with a random word
  - **10%** are kept unchanged
4. Train the model to predict the original tokens

### Why the 80/10/10 split?

If we always used [MASK], the model would learn to only pay attention when it sees [MASK] — but [MASK] never appears in real text during fine-tuning. The random replacement and unchanged tokens force the model to maintain good representations for *all* tokens, not just masked ones.

# MLM step by step

Walkthrough: masking 'The quick brown fox jumps over the lazy dog'

```
1 # Step 1: Select tokens for prediction (15% of 9 tokens ≈ 1-2)
2 tokens = ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]
3 # Randomly select: "quick" (idx 1) and "over" (idx 5)
4
5 # Step 2: Apply the 80/10/10 strategy
6 # "quick" → 80% chance → replaced with [MASK]
7 # "over" → 10% chance → replaced with random word "under"
8
9 # Step 3: Create training example
10 input: "The [MASK] brown fox jumps under the lazy dog"
11 labels: [-1, "quick", -1, -1, -1, "over", -1, -1, -1]
12 # -1 means no loss computed at this position
13
```

continued...



# MLM step by step

Walkthrough: masking 'The quick brown fox jumps over the lazy dog'

- |    |                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------|
| 15 | $P(\text{"quick"} \mid \text{context}) \rightarrow \text{high}$ (adjective slot between "The" and "brown")  |
| 16 | $P(\text{"over"} \mid \text{context}) \rightarrow \text{high}$ (preposition slot between "jumps" and "the") |

...continued

## Key insight

The model must understand both syntax and semantics to predict masked words — it learns deep language representations as a side effect of this objective.

# Next sentence prediction

## BERT's second pre-training objective

**Task:** Given two sentences A and B, predict whether B actually follows A in the original text.

- **50% of the time:** B is the real next sentence (label: `IsNext` )
- **50% of the time:** B is a random sentence from the corpus (label: `NotNext` )

The `[CLS]` token representation is used for this binary classification.

## NSP is controversial

Later work (RoBERTa, 2019) showed that removing NSP actually *improves* performance. The task may be too easy — distinguishing topic is simpler than understanding sentence relationships. ALBERT replaced NSP with the harder **Sentence Order Prediction (SOP)** task.



# BERT architecture variants

## Two model sizes

| Component           | BERT-Base          | BERT-Large         |
|---------------------|--------------------|--------------------|
| Transformer layers  | 12                 | 24                 |
| Hidden size         | 768                | 1024               |
| Attention heads     | 12                 | 16                 |
| Feed-forward size   | 3072               | 4096               |
| Total parameters    | 110M               | 340M               |
| Max sequence length | 512 tokens         | 512 tokens         |
| Vocabulary          | 30,000 (WordPiece) | 30,000 (WordPiece) |

## Special tokens

- **[CLS]**: Classification token (first position, used for sequence-level tasks)
- **[SEP]**: Separator token (between sentence pairs)
- **[MASK]**: Mask token (for MLM pre-training)
- **[PAD]**: Padding token (for batching variable-length sequences)

# BERT input representation

## Three embeddings summed together

BERT's input is the element-wise sum of three embedding types, each producing a 768-dimensional vector:

1. **Token embeddings:** WordPiece vocabulary lookup (30K learned vectors)
2. **Segment embeddings:** Which sentence — A or B? (2 learned vectors)
3. **Position embeddings:** Learned position encoding for positions 0–511

## Input representation for a sentence pair

```
1 sentence_a = "My dog is cute"
2 sentence_b = "He likes playing"
3
4 tokens      = ["[CLS]", "my", "dog", "is", "cute", "[SEP]",
5               "he", "likes", "playing", "[SEP]"]
6 token_emb   = [E_CLS,  E_my, E_dog, E_is, E_cute, E_SEP,
7               E_he, E_likes, E_playing, E_SEP]
8 segment_emb = [E_A,    E_A,  E_A,   E_A,  E_A,    E_A,
```

# BERT input representation

Input representation for a sentence pair

```
9 # Final input = token_emb + segment_emb + position_emb (element-  
   wise)
```

...continued

# WordPiece tokenization

## How BERT handles unknown words

```
1 from transformers import BertTokenizer
2 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
3
4 # Common words stay intact
5 tokenizer.tokenize("The cat sat on the mat")
6 # → ['the', 'cat', 'sat', 'on', 'the', 'mat']
7
8 # Rare words get split into subwords
9 tokenizer.tokenize("unbelievably")
10 # → ['un', '##believable', '##ly'] # "##" means
    continuation
11
12 tokenizer.tokenize("ChatGPT is transformative")
13 # → ['chat', '##g', '##pt', 'is', 'transform', '##ative']
```

# BERT pre-training

Massive-scale pre-training on unlabeled text

## Pre-training data:

- **BooksCorpus:** 800M words (11,038 unpublished books)
- **English Wikipedia:** 2,500M words (text only, no tables/lists/headers)
- **Total:** 3.3 billion words of diverse, high-quality text

## Training details:

- Batch size: 256 sequences (128,000 tokens per batch)
- Training steps: 1 million
- Optimizer: Adam (lr =  $1e-4$ , warmup over first 10K steps)
- Hardware: 4–16 Cloud TPUs
- Training time: ~4 days for BERT-Base

## Key insight

Pre-training learns *general language understanding* that transfers to many downstream tasks. The enormous cost is paid once — fine-tuning is cheap!

# The pre-train then fine-tune paradigm

## Two-stage training

### Stage 1 — Pre-training (done once, expensive):

- Data: 3.3B words of unlabeled text
- Objective: MLM + NSP
- Cost: Days on TPU clusters
- Result: General-purpose language representations

### Stage 2 — Fine-tuning (per task, cheap):

- Data: 1K–100K labeled examples for your specific task
- Objective: Task-specific loss (e.g., cross-entropy for classification)
- Cost: Hours on a single GPU
- Result: Task-specialized model with strong performance

## Why this works

Pre-training captures syntax, semantics, and world knowledge from massive text. Fine-tuning teaches the model to apply that knowledge to a specific task format.



# Fine-tuning for different tasks

## Minimal architecture changes needed

| Task type                      | Input format                          | Output                          | Example                    |
|--------------------------------|---------------------------------------|---------------------------------|----------------------------|
| Single sentence classification | [CLS] sentence [SEP]                  | [CLS] → classifier              | Sentiment analysis         |
| Sentence pair classification   | [CLS] sent-A [SEP]<br>sent-B [SEP]    | [CLS] → classifier              | Natural language inference |
| Question answering             | [CLS] question [SEP]<br>passage [SEP] | Token-level start/end positions | SQuAD                      |
| Token classification           | [CLS] sentence [SEP]                  | Each token → classifier         | Named entity recognition   |

The same pre-trained BERT is used for all tasks — only a simple output layer is added on top.

# Fine-tuning BERT in Python

Using HuggingFace Transformers for sentiment classification

```
1  from transformers import BertForSequenceClassification, Trainer,  
   TrainingArguments  
2  
3  # Load pre-trained BERT with a classification head  
4  model = BertForSequenceClassification.from_pretrained(  
5      'bert-base-uncased',  
6      num_labels=2  # Binary classification (positive/negative)  
7  )  
8  
9  # Define training arguments  
10 training_args = TrainingArguments(  
11     output_dir='./results',  
12     num_train_epochs=3,  
13     per_device_train_batch_size=16,  
14     learning_rate=2e-5,      # Much lower than pre-training!
```

continued...

# Fine-tuning BERT in Python

Using HuggingFace Transformers for sentiment classification

```
15         warmup_steps=500,  
16     )  
17  
18     # Train  
19     trainer = Trainer(  
20         model=model,  
21         args=training_args,  
22         train_dataset=train_dataset,  
23         eval_dataset=eval_dataset,  
24     )  
25     trainer.train()
```

...continued

# Contextual embeddings in action

The word 'bank' gets different embeddings depending on context

```
1 from transformers import BertTokenizer, BertModel
2 import torch
3
4 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5 model = BertModel.from_pretrained('bert-base-uncased')
6
7 sent1 = "I deposited money at the bank"    # Financial
8      institution
9
10 sent2 = "We sat by the river bank"         # Riverbank
11
12 def get_embedding(sentence, target_word):
13     inputs = tokenizer(sentence, return_tensors='pt')
14     outputs = model(**inputs)
```

continued...

# Contextual embeddings in action

The word 'bank' gets different embeddings depending on context

```
13     tokens = tokenizer.tokenize(sentence)
14     idx = tokens.index(target_word) + 1    # +1 for [CLS]
15     return outputs.last_hidden_state[0, idx, :]
16
17     emb1 = get_embedding(sent1, "bank") # Financial context
18     emb2 = get_embedding(sent2, "bank") # River context
19
20     similarity = torch.cosine_similarity(emb1, emb2, dim=0)
21     print(f"Similarity: {similarity:.3f}") # ~0.3-0.5 (low! different
meanings)
```

...continued

# Static vs contextual embeddings

BERT captures meaning differences that static embeddings miss

| Word pair                        | Word2Vec similarity | BERT similarity |
|----------------------------------|---------------------|-----------------|
| bank (financial) vs bank (river) | 1.00 (same vector!) | ~0.42           |
| bank (financial) vs money        | 0.65                | ~0.78           |
| bank (river) vs shore            | 0.52                | ~0.81           |

## Why this matters

Word2Vec and GloVe assign a *single* vector to each word, regardless of context. BERT generates a *different* vector for each occurrence of a word, shaped by its surrounding context. This is why BERT excels at tasks requiring disambiguation — it actually "sees" the difference between financial banks and riverbanks.



# BERT's benchmark results

State-of-the-art on 11 NLP tasks when released (2018)

| Task                                  | Metric   | Previous SOTA | BERT-Large |
|---------------------------------------|----------|---------------|------------|
| SQuAD 2.0 (question answering)        | F1       | 66.3          | 83.1       |
| MNLI (natural language inference)     | Accuracy | 80.6          | 86.7       |
| SST-2 (sentiment analysis)            | Accuracy | 93.2          | 94.9       |
| CoNLL-2003 (named entity recognition) | F1       | 92.6          | 92.8       |

## Key observations:

- Largest gains on tasks requiring deep understanding (QA, NLI)
- Improvements even on well-studied, heavily-optimized benchmarks
- BERT-Large consistently outperforms BERT-Base

## Impact

BERT made pre-trained transformers the default starting point in NLP. Nearly all subsequent models — RoBERTa, ALBERT, DistilBERT, GPT-2 — build on ideas BERT popularized.

# What does BERT learn?

## Probing BERT's internal representations

Research has shown that BERT's layers form a linguistic processing pipeline:

1. **Lower layers (1–4):** Surface features — part-of-speech tags, word boundaries, morphology
2. **Middle layers (5–8):** Syntactic structure — parse trees, dependency relations, subject-verb agreement
3. **Upper layers (9–12):** Semantics and pragmatics — word sense disambiguation, coreference, entity types
4. **Final layers:** Task-specific representations that emerge during fine-tuning

## Analogy to vision

This is similar to how convolutional neural networks learn: early layers detect edges, middle layers detect shapes, and later layers detect objects. BERT does the same thing with language — from characters to meaning.

# Layer analysis with probing

## Extracting representations from different layers

```
1  from transformers import BertModel
2
3  model = BertModel.from_pretrained('bert-base-uncased',
4  output_hidden_states=True)
5  outputs = model(**inputs)
6  hidden_states = outputs.hidden_states # 13 tensors:
7  embedding + 12 layers
8
9  # Results from probing studies (Tenney et al., 2019):
10 layer_specialization = {
11     "Layers 0-2": ["POS tagging", "Word boundaries"],
12     # Surface
13     "Layers 3-6": ["Parse trees", "Dependencies"],
14     # Syntax
15     "Layers 7-9": ["Semantic roles", "Coreference"],
16     # Semantics
```

# Discussion

## Questions to consider

1. **MLM vs autoregressive modeling:** Why is MLM better for *understanding* tasks? Could BERT generate text like GPT? What are the trade-offs?
2. **The 80/10/10 masking strategy:** Why not use 100% [MASK] replacement? What problem does the random word replacement solve? Could we improve this strategy?
3. **Pre-training data choices:** Why use books and Wikipedia? Would social media text work as well? How does data quality affect what BERT learns?
4. **Fine-tuning efficiency:** Why does fine-tuning work so well with so little data? When might it fail? How much labeled data do we actually need?

# References

## Further reading

**Devlin et al. (2019, *NAACL*)** "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" — The original BERT paper.

**Tenney et al. (2019, *ACL*)** "BERT Rediscovered the Classical NLP Pipeline" — Layer-by-layer analysis of what BERT learns.

**Clark et al. (2019, *BlackboxNLP*)** "What Does BERT Look At? An Analysis of BERT's Attention" — Attention pattern analysis.

**HuggingFace Course, Chapter 1** — Practical introduction to transformer models.

**Jay Alammar: The Illustrated BERT** — Visual walkthrough of BERT's architecture.



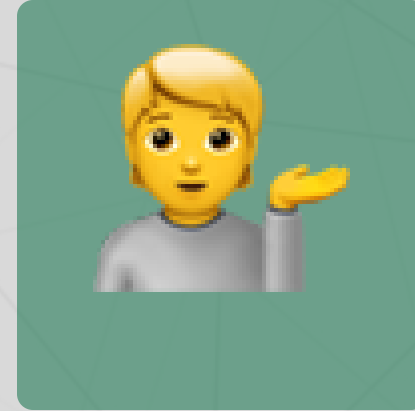
# Questions?



Email



Discord



Office hours

Up next...

BERT variants: how RoBERTa, ALBERT, DistilBERT, and ELECTRA improved on the original